

tuple 정리

세 개 이상의 값을 하나로 묶을 수 있는 표준 타입입니다.

사용처 (Where)

- 세 값 이상을 임시로 묶을 때
- 함수에서 여러 값을 반환할 때
- 정렬 키가 여러 개일 때

방법 (How to Use)

- make_tuple 또는 중괄호로 생성
- get<인덱스>(t)로 접근
- structured binding으로 분해
- 의미가 복잡하면 struct가 더 좋음

기본 정보

- 필요 헤더: #include <tuple>
- 기본 선언: tuple<int, string, double> t;

왜 써야 할까? (Why)

- pair보다 많은 값을 간단히 묶음
- 여러 반환값을 표현하기 쉬움
- 정렬 가능한 임시 데이터 구성에 유용

주요 함수

함수/문법	의미
make_tuple	tuple 생성
get<0>(t)	0번째 값
tie	값 분해
structured binding	auto [a,b,c]

기본 코드 예시

```
#include <iostream>
#include <tuple>
using namespace std;

int main() {
    tuple<int, string, double> t = {1, "Kim", 3.5};
    cout << get<0>(t) << ' ' << get<1>(t);
}
```

tuple 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
생성/접근	$O(1)$
비교	앞 원소부터 순서대로
정렬	$O(n \log n)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
표현력	의도를 타입으로 직접 표현	구조체/클래스를 직접 작성	배열/struct 수동 관리
코드 길이	짧고 표준적인 형태	필드와 생성자 작성 필요	초기화와 접근 실수 가능
유지보수	다른 STL과 자연스럽게 연동	설계가 늘어날수록 관리 필요	가독성이 떨어지기 쉬움
추천 상황	간단한 값 묶음/조건 전달	도메인 의미가 큰 객체	C API와 맞춰야 할 때

응용: 여러 값 반환

함수에서 번호, 이름, 점수를 tuple로 반환하고 structured binding으로 받습니다.

```
#include <iostream>
#include <tuple>
using namespace std;

tuple<int, string, int> makeStudent() {
    return {1, "Kim", 90};
}

int main() {
    auto [id, name, score] = makeStudent();
    cout << id << ' ' << name << ' ' << score;
}
```

처음 배울 때 자주 하는 실수

- `get<0>`, `get<1>`만 많아지면 의미가 흐려짐
- 도메인 객체는 struct가 더 읽기 쉬움
- 인덱스는 컴파일 타임 상수여야 함

비슷한 항목과 차이

- pair는 두 값만 묶음
- struct는 필드 이름이 있어 가독성이 좋음
- array는 같은 타입 여러 개를 묶음

한 줄 정리: tuple은(는) '세 값 이상을 임시로 묶을 때' 상황에서 먼저 떠올리면 좋은 STL입니다.